

CKA: Scheduling

KodeKloud Complete Study Notes — Certified Kubernetes Administrator

Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

Scheduling Section Introduction

This article provides a comprehensive lesson on Kubernetes scheduling concepts and techniques for managing pods and workloads effectively.

Hello, my name is Mumshad Mannambeth, and I welcome you to this comprehensive lesson on scheduling in Kubernetes. In this module, you will explore essential scheduling concepts and techniques that empower you to manage pods and workloads effectively. Previously, we briefly introduced the installation and configuration of the Kubernetes scheduler. Now, we will dive deeper into the various customization options that influence scheduler behavior through a series of practical, hands-on labs.

Overview

In this lesson, you will learn about the following topics:

- **Manual Scheduling:** Understand how to manually schedule pods to specific nodes.
- **DaemonSets:** Discover how to run a copy of a pod on each node in your cluster.
- **Labels and Selectors:** Learn how to use labels and selectors to manage and target specific groups of pods.
- **Resource Requests and Limits:** See how resource constraints affect scheduling decisions.
- **Multiple Schedulers:** Explore configuration strategies for running multiple schedulers within a cluster.
- **Viewing Scheduler Events:** Learn how to monitor and troubleshoot scheduler events for optimal performance.

Each section of this lesson is complemented by hands-on labs designed to reinforce your understanding and give you real-world experience with Kubernetes scheduling.

Let's get started with the first topic—Manual Scheduling—and progressively build your expertise in Kubernetes scheduling.

Course Objectives

Core Concepts

Scheduling

- Manual Scheduling
- Labels & Selectors
- Resource Limits
- Daemon Sets
- Multiple Schedulers
- Scheduler Events
- Configure Kubernetes Scheduler

Logging Monitoring

Application Lifecycle Management

Cluster Maintenance

Security

Storage

Networking

Installation, Configuration & Validation

Troubleshooting

Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

Manual Scheduling

This guide explains how to assign pods to nodes without relying on Kubernetes' built-in scheduler for tighter control over pod placement.

Welcome to this lesson on manually scheduling pods in Kubernetes. This guide explains how to assign pods to nodes without relying on Kubernetes' built-in scheduler. Manual scheduling can be useful in niche scenarios where you need tighter control over pod placement. In this article, we review a basic pod manifest, demonstrate how manual scheduling works, and show you how to use binding objects to reassign pods if necessary.

Understanding the Default Scheduler Behavior

Every pod definition includes a field called `nodeName`, which is left unset by default. The Kubernetes scheduler automatically scans for pods without a `nodeName` and selects an appropriate node by updating this field and creating a binding object. Consider the basic pod manifest below:

```
```yaml theme={null} apiVersion: v1 kind: Pod metadata: name: nginx labels: name: nginx spec: containers: - name: nginx image: nginx ports: - containerPort: 8080
```

Typically, you do not include the `nodeName` field in your manifest. The scheduler uses this field only at

## Manually Setting the Node Name

To manually assign a pod to a specific node during creation, populate the `nodeName` field in the manifest

```
```yaml theme={null}
apiVersion: v1
kind: Pod
metadata:
  name: nginx
  labels:
    name: nginx
spec:
  nodeName: node02
  containers:
  - name: nginx
    image: nginx
    ports:
    - containerPort: 8080
```

After creating the pod with this manifest, check its status with:

```
```bash theme={null} kubectl get pods
```

Sample output before the scheduler assigns the IP:

```
```bash theme={null}
NAME      READY   STATUS    RESTARTS   AGE
nginx     0/1     Pending   0           3s
```

And after the pod becomes ready:

```
bash theme={null} kubectl get pods NAME READY STATUS RESTARTS AGE IP NODE nginx 1/1 Running 0 9s
10.40.0.4 node02
```

The `nodeName` must be set during pod creation. Once the pod is running, Kubernetes does not permit modifications to the `nodeName` field.

Reassigning a Running Pod Using a Binding Object

If a pod is already running and you need to change its node assignment, you cannot modify its `nodeName` directly. In this scenario, you can create a binding object that mimics the scheduler's behavior.

1. Create a binding object that specifies the target node ("node02"):

```
yaml theme={null} apiVersion: v1 kind: Binding metadata: name: nginx target: apiVersion: v1 kind:
Node name: node02
```

1. The original pod definition remains unchanged:

```
yaml theme={null} apiVersion: v1 kind: Pod metadata: name: nginx labels: name: nginx spec:
containers: - name: nginx image: nginx ports: - containerPort: 8080
```

1. Convert the YAML binding to JSON (e.g., save it as `binding.json`) and send a POST request to the pod's binding API using curl:

```
bash theme={null} curl --header "Content-Type: application/json" --request POST --data
@binding.json http://$SERVER/api/v1/namespaces/default/pods/nginx/binding
```

This binding instructs Kubernetes to assign the existing pod to the specified node without altering its original manifest.

Quick Reference Table

Method	Use Case	Example Snippet Reference
Direct Assignment	Set <code>nodeName</code> during pod creation	See manifest with <code>nodeName: node02</code>
Using a Binding Object	Reassign a running pod	See binding object example

Additional Resources

- [Kubernetes Basics](#)
- [Kubernetes Documentation](#)

That's it for this lesson. Experiment with manually scheduling pods to solidify your understanding of Kubernetes node assignments. Happy scheduling!

Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/lms.txt> Use this file to discover all available pages before exploring further.

Labels and Selectors

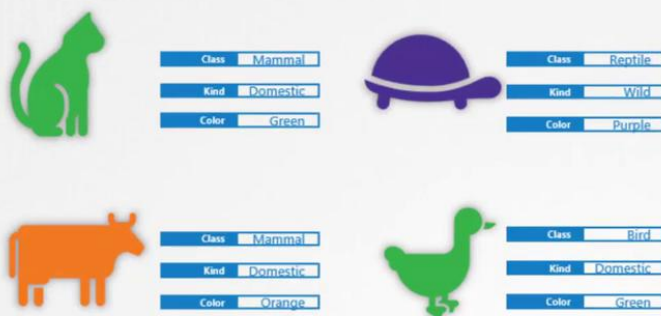
This article explores labels, selectors, and annotations in Kubernetes for effective resource management and organization.

Welcome to this comprehensive guide on labels, selectors, and annotations in Kubernetes. In this article, we will explore how labels and selectors help group and filter items effectively, and we will also discuss how annotations are used to store additional metadata. By the end, you'll understand how these key concepts are applied to manage resources in a Kubernetes environment.

Introduction to Labels and Selectors

Labels and selectors offer a systematic approach to categorizing items. Think of a scenario where you have various species of animals, and you need to filter them based on attributes such as class, kind, or color. For example, you might want to filter for all green animals or just the green birds. By attaching labels like class, kind, and color to each animal, you can later use selectors to efficiently filter and display the items that meet your criteria.

Selectors



In everyday applications, labels and selectors are ubiquitous. They function similarly to keywords in YouTube videos or blog posts, aiding users in finding relevant content. In online stores, labels empower customers to filter products based on various characteristics.

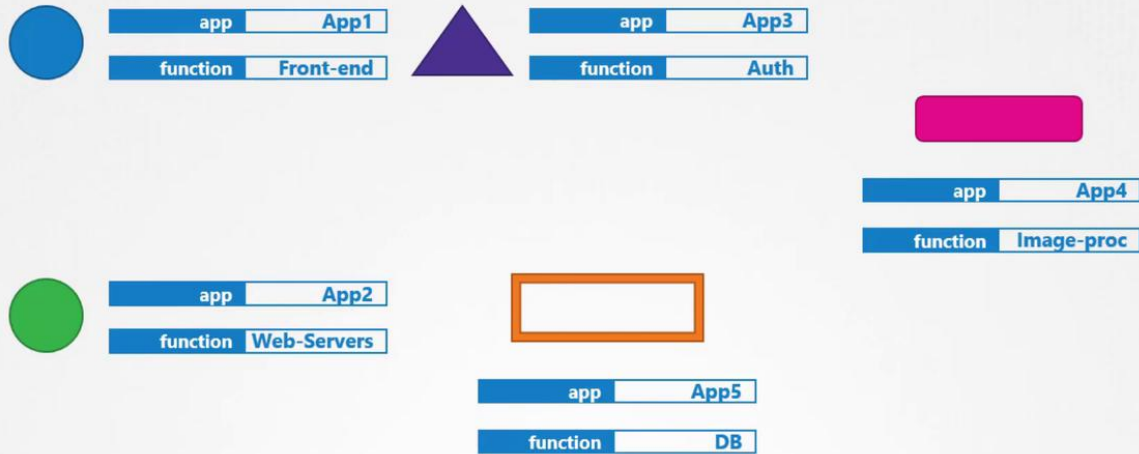
Labels and Selectors in Kubernetes

In Kubernetes, labels and selectors are instrumental in managing an array of objects such as Pods, Services, ReplicaSets, and Deployments. As the number of objects in a cluster grows, these tools become essential for grouping and selecting objects by application, functionality, or type.



For instance, you might attach labels like "app" or "function" to your Kubernetes objects and later use selectors to filter objects based on specific conditions (e.g., app equals "App1").

Labels



Specifying Labels in Kubernetes

To apply labels to a Kubernetes object such as a Pod, include a `labels` section under the `metadata` field in its definition file. Consider the following Pod definition example:

```
```yaml
theme={null}
apiVersion: v1
kind: Pod
metadata:
 name: simple-webapp
 labels:
 app: App1
 function: Front-end
spec:
 containers:
 - name: simple-webapp
 image: simple-webapp
 ports:
 - containerPort: 8080
```
```

After creating the Pod, you can retrieve it using the `kubectl get pods` command with a selector. For example:

```
```bash
theme={null}
kubectl get pods --selector app=App1
NAME READY STATUS RESTARTS AGE
simple-webapp 0/1 Completed 0 1d
```
```

Using selectors with `kubectl` commands helps you filter and manage resources in large clusters with ease.

Using Labels and Selectors with ReplicaSets

In Kubernetes, internal mechanisms utilize labels and selectors to connect different objects. When creating a ReplicaSet to manage three Pods, you first label the Pod definitions and then use a selector in the ReplicaSet definition to ensure the correct Pods are grouped together.

A ReplicaSet definition includes labels in two key areas:

1. Within the ReplicaSet's metadata (allowing other objects to reference the ReplicaSet).
2. Within the `template` of the ReplicaSet's specification (applying the labels to the Pods).

By setting the `selector` field in the ReplicaSet specification to match the labels defined on the Pods, you ensure that the ReplicaSet manages the intended Pods. Below is an example configuration:

```
```yaml
theme={null}
apiVersion: apps/v1
kind: ReplicaSet
metadata:
 name: simple-webapp
 labels:
 app: App1
 function: Front-end
spec:
 replicas: 3
 selector:
 matchLabels:
 app: App1
 template:
 metadata:
 labels:
 app: App1
 function: Front-end
 spec:
 containers:
 - name: simple-webapp
 image: simple-webapp
```
```

<Callout icon="lightbulb" color="#1CB2FE">

If you require more granular control for selecting Pods, you can list multiple labels in the `matchLabels` field.

Annotations

Annotations differ from labels and selectors in that they are used to store additional metadata that is not used for selection.

```
```yaml
theme={null}
apiVersion: apps/v1
kind: ReplicaSet
metadata:
 name: simple-webapp
 labels:
 app: App1
 function: Front-end
 annotations:
 buildversion: "1.34"
spec:
 replicas: 3
 selector:
 matchLabels:
 app: App1
 template:
 metadata:
 labels:
 app: App1
 function: Front-end
 spec:
 containers:
 - name: simple-webapp
 image: simple-webapp
```
```

When the ReplicaSet is created, it matches the Pods based on labels, ensuring that only the intended Pods are managed. The same mechanism is used when creating Services, where the Service's selector matches the labels set on the Pods.

Conclusion

This article has provided an in-depth look at labels, selectors, and annotations in Kubernetes. These concepts are essential for effectively managing and grouping objects within your clusters. For further hands-on practice, navigate to the coding exercises section and start working with labels and selectors today.

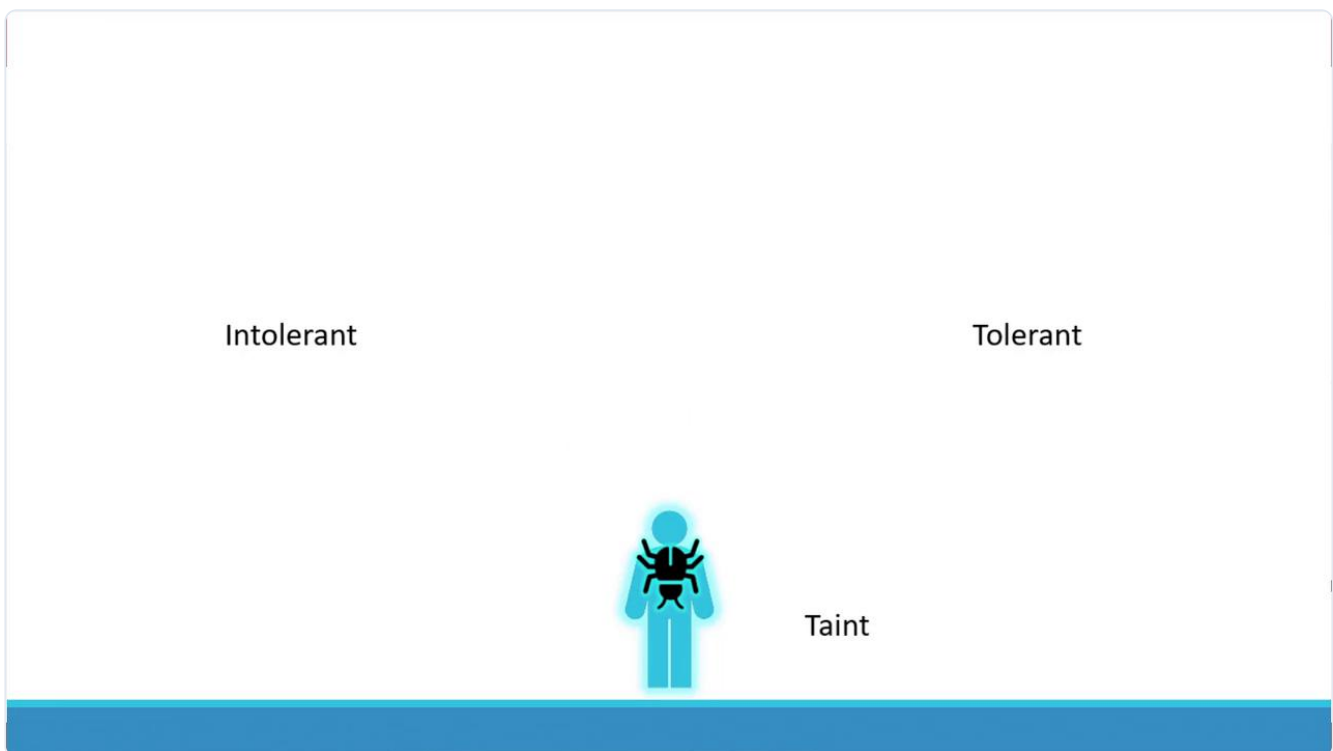
Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/lms.txt> Use this file to discover all available pages before exploring further.

Taints and Tolerations

This article explores how taints and tolerations control pod scheduling on specific nodes in Kubernetes.

In this article, we explore the relationship between pods and nodes, and demonstrate how taints and tolerations control pod scheduling on specific nodes. Think of it like using a repellent on a person to keep bugs away. In Kubernetes, a taint is applied to a node (the repellent) to repel pods unless they have a matching toleration (an immunity to the repellent). This ensures that only intended pods can run on tainted nodes.



How Taints and Tolerations Work

Two factors determine whether a pod can run on a node:

1. The taint applied on the node.
2. The pod's toleration for that taint.

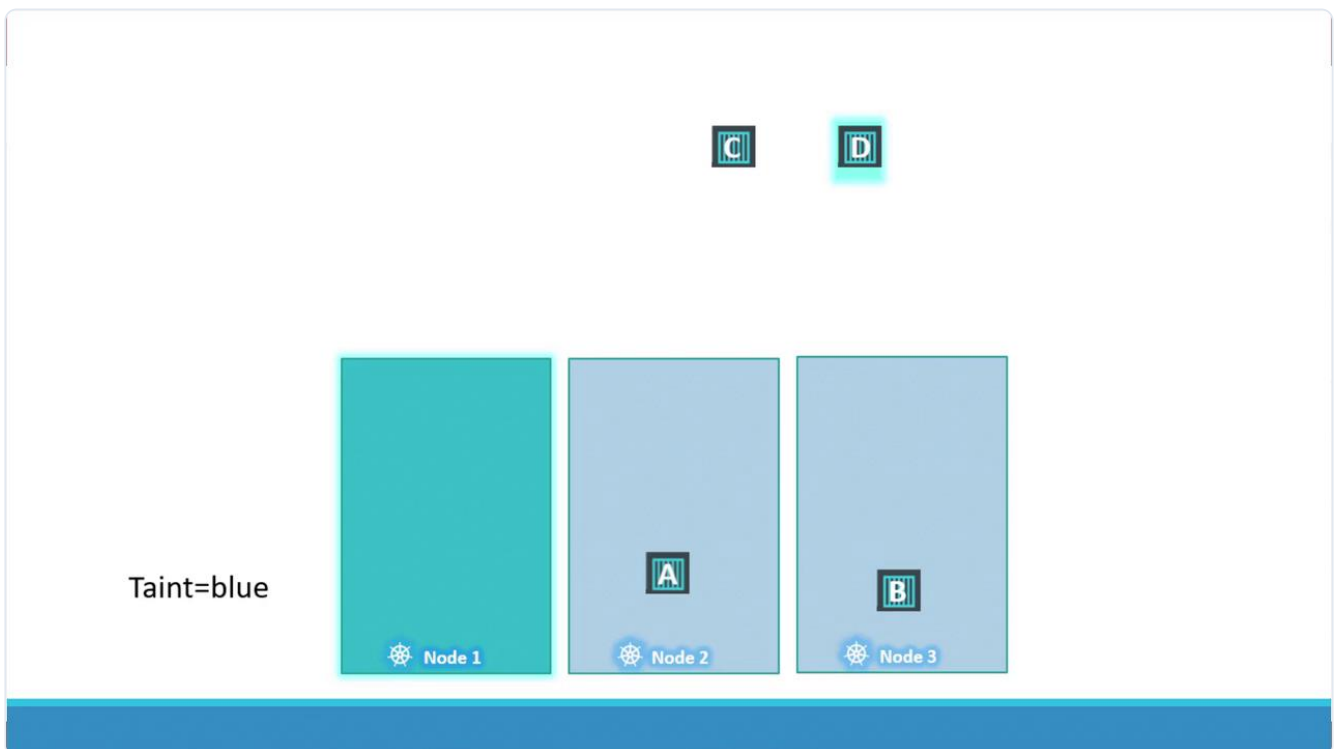
In Kubernetes, taints and tolerations are used solely for scheduling control, not security. Consider a simple cluster with three worker nodes (node one, two, and three) and four pods (A, B, C, and D). By default, without any taints or tolerations, the scheduler distributes the pods evenly across all nodes.

Suppose you want to dedicate node one to a specific application:

- First, taint node one (using, for example, the key-value pair "app=blue") so that no pods are scheduled there by default.
- Then, add an appropriate toleration to the dedicated pod (Pod D) so that it alone can run on node one.

When the scheduler assigns pods:

- Pod A, lacking a toleration, will be scheduled on node two.
- Pod B is repelled from node one and lands on node three.
- Pod C, without a matching toleration, is scheduled on node two or three.
- Pod D, having the correct toleration, is scheduled on node one.



Tainting a Node

To taint a node, use the following command where you specify the node name along with a taint in the format of a key-value pair and taint effect:

```
```bash theme={null} kubectl taint nodes node-name key=value:taint-effect
```

For example, to taint node one so that it only accepts pods associated with the application blue, run:

```
```bash theme={null}
kubectl taint nodes node1 app=blue:NoSchedule
```

There are three taint effects:

- **NoSchedule:** Pods that do not tolerate the taint will not be scheduled on the node.
- **PreferNoSchedule:** The scheduler tries to avoid placing non-tolerating pods on the node, but it is not strictly enforced.
- **NoExecute:** New pods without a matching toleration will not be scheduled, and existing pods will be evicted from the node.

Adding Tolerations to Pods

To allow a specific pod to run on a tainted node, add a toleration to the pod's manifest. For example, to enable Pod D to run on node one (tainted with "app=blue:NoSchedule"), modify the pod definition as follows:

```
```yaml
theme={null}
apiVersion: v1
kind: Pod
metadata:
 name: myapp-pod
spec:
 containers:
 - name: nginx
 image: nginx
 tolerations:
 - key: "app"
 operator: "Equal"
 value: "blue"
 effect: "NoSchedule"
```
```

This toleration lets the pod bypass the taint and be scheduled on node one.

Understanding the NoExecute Effect

Using the **NoExecute** taint effect has a more immediate impact. Suppose node one is tainted with the key

- * Pods without a matching toleration (e.g., Pod C) will be evicted from node one if already running, or prevented from being scheduled.
- * Pods with the correct toleration (e.g., Pod D) continue to run on node one.

<Frame>

! [The image illustrates Kubernetes nodes with a "NoExecute" taint, showing pods A, B, C, and D distributed across nodes.]

<Callout icon="lightbulb" color="#1CB2FE">

Taints and tolerations are used to ensure that pods are not accidentally scheduled on nodes that should not run them.

Master Node Taints

While the discussion so far has focused on worker nodes, master nodes are also capable of running pods. By

```
```bash
theme={null}
kubectl describe node kubemaster | grep Taint
```
```

The output will display a taint similar to:

```
text theme={null} Taints: node-role.kubernetes.io/master:NoSchedule
```

This configuration follows best practices by ensuring that only essential components run on the master node.

Summary

Taints and tolerations provide a powerful mechanism for controlling pod scheduling in Kubernetes clusters:

| Feature | Description | Example Command or Manifest Snippet |
|-------------------------|---|---|
| Tainting a Node | Prevents pods from being scheduled on a node unless they tolerate the taint. | <code>kubectl taint nodes node1 app=blue:NoSchedule</code> |
| Toleration in a Pod | Allows a pod to run on a tainted node by including a toleration in its manifest. | See the YAML snippet above |
| Taint Effect: NoExecute | Evicts pods not tolerating the taint from a node and prevents new non-tolerant pods from joining. | Command similar to the taint command with <code>NoExecute</code> effect |

Now that you understand taints and tolerations, experiment with their configuration in your Kubernetes cluster to better control pod placement. For more details on advanced scheduling techniques, check out [Kubernetes Documentation](#).

Remember that taints and tolerations only influence pod scheduling. If you need to enforce pod placement on specific nodes, use node affinity rules.

Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

Node Selectors

This lesson covers how to use node selectors in Kubernetes to ensure specific pods run on designated nodes for better performance and resource management.

Welcome to this lesson on node selectors in Kubernetes. In this guide, you'll learn how to ensure that specific pods run only on designated nodes within your cluster. Node selectors help align your pod deployments with the underlying hardware characteristics of your nodes, enhancing performance and resource management.

Imagine managing a three-node cluster where two nodes have limited hardware resources, and one node is equipped with higher resources. Different workloads run in your cluster, and data processing tasks that demand more computing power should ideally be scheduled on the larger node. Without any scheduling constraints, any pod might land on any node—even on one with insufficient resources—leading to performance bottlenecks.

Node selectors restrict pod placement by matching key-value pairs defined in the pod's specification against the labels on the nodes.

Configuring Node Selectors

To ensure that a pod is restricted to run on a specific node, you can modify the pod's definition file using node selectors. Below is an example of a pod definition YAML file that deploys a data processing image exclusively on a node labeled as "Large":

```
```yaml theme={null} apiVersion: v1 kind: Pod metadata: name: myapp-pod spec: containers: - name: data-processor image: data-processor nodeSelector: size: Large
```

In this configuration, the Kubernetes scheduler identifies the appropriate node by matching the label with

```
Labeling a Node
```

Before deploying your pod, you must label your node so that it can be recognized by the selector Use the

```
```bash theme={null}
kubectl label nodes node-1 size=Large
```

Once the node is labeled, create the pod by applying your configuration:

```
bash theme={null} kubectl create -f pod-definition.yaml
```

Kubernetes will then schedule your pod on the node that matches the selector—in this case, the larger node.

Limitations and Advanced Scheduling

While node selectors are ideal for simple scenarios involving a single label, they come with certain limitations. For instance, if you need to schedule a pod on a node that is either large or medium, or on any node that is not labeled as small, a basic node selector may not suffice. In these cases, consider using node affinity and anti-affinity features, which offer advanced scheduling capabilities to define more complex placement rules.

Make sure that your nodes are pre-labeled with the correct key-value pairs before deploying your pods. Failure to do so may prevent the scheduler from matching pods to the intended nodes.

Further Resources

- [Kubernetes Documentation](#)
- [Kubernetes Basics](#)

Happy learning!

Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/lms.txt> Use this file to discover all available pages before exploring further.

Node Affinity

This article explains node affinity in Kubernetes, detailing advanced scheduling rules for pod placement based on node labels.

Welcome to this comprehensive lesson on node affinity in Kubernetes. In this guide, you'll learn how node affinity extends the capabilities of basic node selectors by allowing advanced expressions like In, NotIn, and Exists. This feature enables you to specify detailed rules for pod placement based on node labels.

Previously, you might have used node selectors for basic scheduling. For example, to ensure that a large data processing pod runs on a large node, you could use a configuration like this:

```
```yaml theme={null} apiVersion: v1 kind: Pod metadata: name: myapp-pod spec: containers: - name: data-processor image: data-processor nodeSelector: size: Large
```

While node selectors are simple and intuitive, they lack support for advanced matching operators

```
```yaml theme={null}
apiVersion: v1
kind: Pod
metadata:
  name: myapp-pod
spec:
  containers:
    - name: data-processor
      image: data-processor
  affinity:
    nodeAffinity:
      requiredDuringSchedulingIgnoredDuringExecution:
        nodeSelectorTerms:
          - matchExpressions:
              - key: size
                operator: In
                values:
                  - Large
```

- The `affinity` key under `spec` introduces the `nodeAffinity` configuration.
- The field `requiredDuringSchedulingIgnoredDuringExecution` indicates that the scheduler must place the pod on a node meeting the criteria. Once the pod is running, any changes to node labels are ignored.
- The `nodeSelectorTerms` array contains one or more `matchExpressions`. Each expression specifies a label key, an

operator, and a list of values. Here, the `In` operator ensures that the pod is scheduled only on nodes where the label `size` includes 'Large'.

To allow for more flexible scheduling, such as permitting a pod to run on either large or medium nodes, simply add additional values to the list. Alternatively, you can use the `NotIn` operator to explicitly avoid placing a pod on nodes with specific labels. For example, to avoid nodes labeled as small:

```
```yaml theme={null} apiVersion: v1 kind: Pod metadata: name: myapp-pod spec: containers: - name: data-processor image: data-processor affinity: nodeAffinity: requiredDuringSchedulingIgnoredDuringExecution: nodeSelectorTerms: - matchExpressions: - key: size operator: NotIn values: - Small
```

In cases where you only need to verify the presence of a label without checking for specific values, the

```
```yaml theme={null}
apiVersion: v1
kind: Pod
metadata:
  name: myapp-pod
spec:
  containers:
    - name: data-processor
      image: data-processor
  affinity:
    nodeAffinity:
      requiredDuringSchedulingIgnoredDuringExecution:
        nodeSelectorTerms:
          - matchExpressions:
              - key: size
                operator: Exists
```

Once a pod is scheduled using node affinity rules, these rules are only evaluated during scheduling. Changes to node labels after scheduling will not affect a running pod due to the "ignored during execution" behavior.

There are two primary scheduling behaviors for node affinity:

1. Required During Scheduling, Ignored During Execution

2. The pod is scheduled only on nodes that fully satisfy the affinity rules.
3. Once running, changes to node labels do not impact the pod.

4. Preferred During Scheduling, Ignored During Execution

5. The scheduler prefers nodes that meet the affinity rules but will place the pod on another node if no matching nodes are available.

Node Affinity Types

Available:

requiredDuringScheduling**Ignored**DuringExecution

preferredDuringScheduling**Ignored**DuringExecution

	DuringScheduling	DuringExecution
Type 1	Required	
Type 2	Preferred	

Future enhancements may introduce additional affinity types, such as **Required During Execution**. In this model, if a node's labels change after a pod is running and no longer meet the affinity criteria, the pod would be evicted.

Node Affinity Types

Planned:

requiredDuringScheduling**Required**DuringExecution

preferredDuringScheduling**Required**DuringExecution

	DuringScheduling	DuringExecution
Type 1	Required	Ignored
Type 2	Preferred	Ignored
Type 3	Required	Required
Type 4	Preferred	Required

Summary

Node affinity empowers you to define sophisticated scheduling rules for pod placement based on node labels. Key takeaways include:

- Using `nodeSelectorTerms` with `matchExpressions` to specify rules.
- Leveraging operators such as `In`, `NotIn`, and `Exists` for flexible matching.
- Understanding the scheduling phases: during scheduling and after deployment (execution), and how they interact.

This concludes our lesson on node affinity. Practice the provided configurations and explore further by comparing node affinity with taints and tolerations. Happy learning and coding!

Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/lms.txt> Use this file to discover all available pages before exploring further.

Taints and Tolerations vs Node Affinity

This article explains controlling pod placement in Kubernetes using taints, tolerations, and node affinity for optimal scheduling and exclusive node usage.

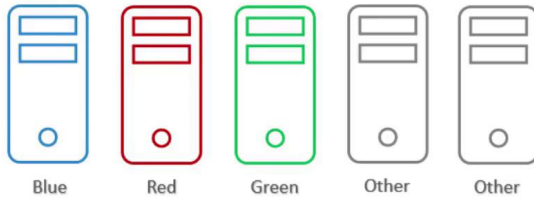
Welcome to this article where we explain how to control pod placement in a Kubernetes cluster by combining taints, tolerations, and node affinity. In our example, we have three nodes and three pods, each identified by a distinct color—blue, red, and green. Our objective is to ensure that each pod is scheduled on the node with the corresponding color while preventing unwanted workloads from running on these dedicated nodes.

In Kubernetes, taints and tolerations are primarily used to repel pods from nodes unless they explicitly tolerate the taint, whereas node affinity is used to attract pods to nodes that satisfy specific label criteria.

Using Taints and Tolerations

To begin, we apply a taint to each node that marks it with its respective color (blue, red, or green). Then, each pod is configured with a corresponding toleration. With this setup, the Kubernetes scheduler places the pods on nodes that accept their tolerations. For instance, the green pod is placed on the green node and the blue pod on the blue node.

Taints and Tolerations



However, while taints and tolerations ensure that pods with matching tolerations are admitted by the nodes, they do not guarantee exclusive scheduling. Consequently, a pod (for example, a red pod) might still be scheduled on an untainted node, leading to undesired placements.

Using Node Affinity

To overcome the limitation of taints and tolerations, we leverage node affinity. This method involves labeling each node with its specific color and then configuring node selectors or advanced affinity rules in the pod specifications. Node affinity ensures a pod lands only on the node with the matching label.

While node affinity directs pods to the correct nodes, it does not restrict other pods from also being scheduled on these nodes. This means that although our desired pods are correctly placed, the nodes might still host pods not meant for them.

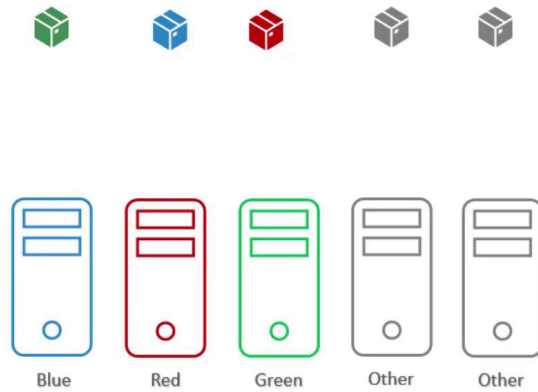
Combining Taints and Tolerations with Node Affinity

For exclusive node usage, combining both strategies is the optimal solution. The integration works as follows:

1. Apply taints on nodes and specify corresponding tolerations in pod configurations to block any pod without the proper toleration.
2. Use node affinity rules to ensure that each pod is only scheduled on a node with a matching label.

This combined approach dedicates the nodes exclusively to the intended pods, assuring correct pod assignments and preventing interference by other workloads.

Taints/Tolerations and Node Affinity



In summary, leveraging both taints/tolerations and node affinity in Kubernetes ensures precise pod scheduling. This approach is particularly useful in multi-tenant clusters where exclusive node usage is critical.

Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

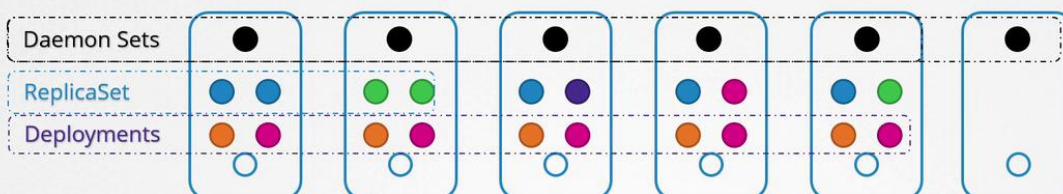
DaemonSets

This comprehensive guide explores DaemonSets in Kubernetes, their use cases, and provides a step-by-step example for deployment in your cluster.

Welcome to this comprehensive guide on DaemonSets in Kubernetes. In this article, we will dive deep into how DaemonSets work, explore their primary use cases, and provide a step-by-step example to help you deploy one in your cluster.

DaemonSets ensure that exactly one copy of a pod runs on every node in your Kubernetes cluster. When you add a new node, the DaemonSet automatically deploys the pod on the new node. Likewise, when a node is removed, the corresponding pod is also removed. This guarantees that a single instance of the pod is consistently available on each node.

I Daemon Sets

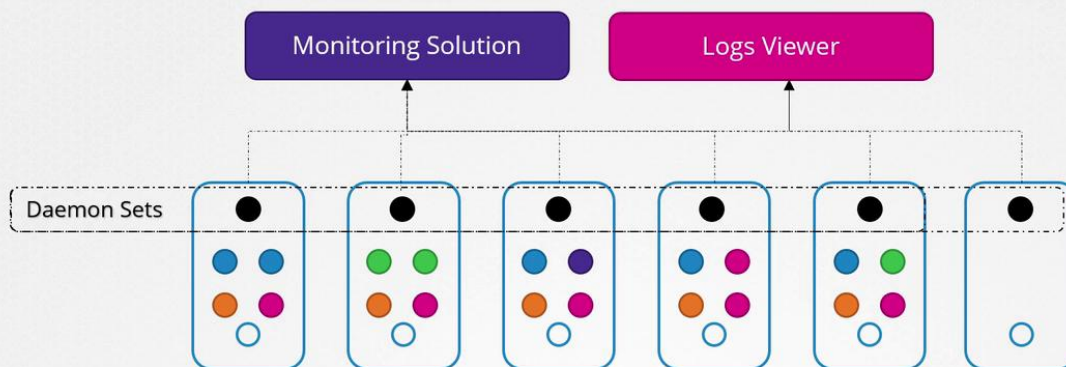


Use Cases for DaemonSets

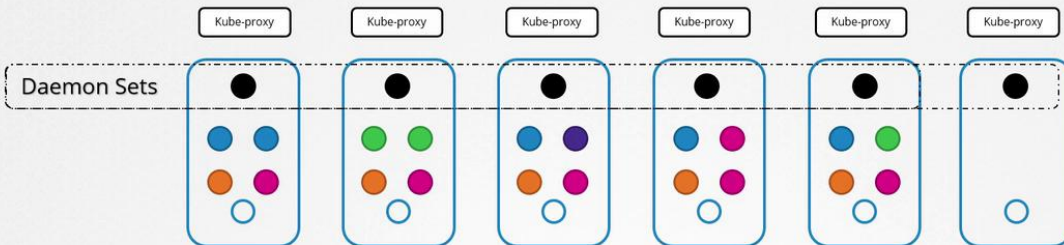
DaemonSets are particularly useful in scenarios where you need to run background services or agents on every node. Some common use cases include:

- **Monitoring agents and log collectors:** Deploy monitoring tools or log collectors across every node to ensure comprehensive cluster-wide visibility without manual intervention.
- **Essential Kubernetes components:** Deploy critical components, such as kube-proxy, which Kubernetes requires on all worker nodes.
- **Networking solutions:** Ensure consistent deployment of networking agents like those used in VNet or weave-net across all nodes.

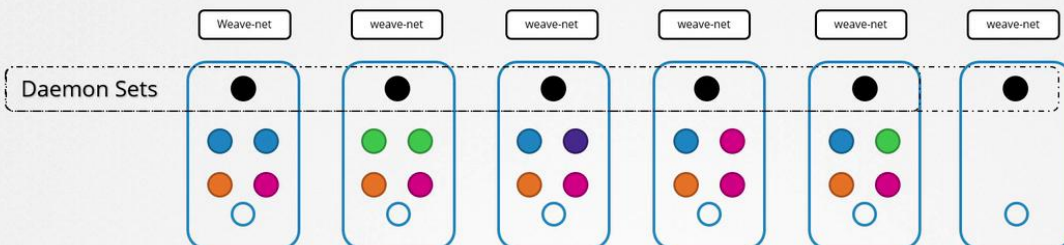
Daemon Sets – UseCase



| Daemon Sets – UseCase – kube-proxy



| Daemon Sets – UseCase – Networking



Creating a DaemonSet

Creating a DaemonSet is analogous to creating a ReplicaSet. The DaemonSet YAML configuration consists of a pod template under the `template` section and a selector that binds the DaemonSet to its pods. A typical DaemonSet definition includes the API version, kind, metadata, and specifications. Note that the API version is `apps/v1` and the kind is set to `DaemonSet`.

Below is an example DaemonSet definition file that deploys a monitoring agent:

```
```yaml theme={null}
```

## daemon-set-definition.yaml

---

```
apiVersion: apps/v1 kind: DaemonSet metadata: name: monitoring-daemon spec: selector: matchLabels: app: monitoring-agent template: metadata: labels: app: monitoring-agent spec: containers: - name: monitoring-agent image: monitoring-agent
```

```
:::note Important
```

```
Ensure that the labels in the selector match those in the pod template. Consistent labeling is crucial for
:::
```

Once your YAML file is ready, create the DaemonSet using the following command:

```
```bash theme={null}  
kubectl create -f daemon-set-definition.yaml
```

Verify the DaemonSet's creation by running:

```
```bash theme={null} kubectl get daemonsets
```

This command produces output similar to:

```
```console theme={null}  
NAME                DESIRED    CURRENT    READY    UP-TO-DATE    AVAILABLE    AGE  
monitoring-daemon    1          1          1        1             1            41
```

For more detailed information on your DaemonSet, use:

```
bash theme={null} kubectl describe daemonset monitoring-daemon
```

How DaemonSets Schedule Pods

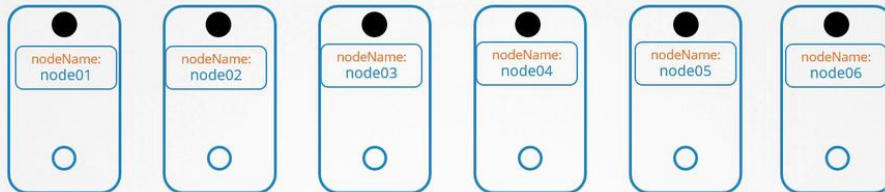
Prior to Kubernetes version 1.12, scheduling a pod on a specific node was often achieved by manually setting the `nodeName` property within the pod specification. However, since version 1.12, DaemonSets leverage the default scheduler in conjunction with node affinity rules. This improvement ensures that a pod is automatically scheduled on every node without manual intervention.

How does it work?



Default Behavior till v1.12

From v1.12 - uses NodeAffinity and default scheduler



Tip DaemonSets are an ideal solution for deploying services that must run on every node, such as monitoring agents and essential networking components. Leveraging node affinity simplifies management as your cluster scales.

Conclusion

DaemonSets provide an efficient mechanism to ensure that key services are uniformly deployed across your Kubernetes cluster. Whether you're running log collectors, monitoring agents, or essential network components like kube-proxy and weave-net, DaemonSets help maintain consistency and reliability in dynamic environments.

For further reading, explore these resources:

- [Kubernetes Documentation](#)
- [DaemonSets Concept](#)

Happy clustering!

Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/lms.txt> Use this file to discover all available pages before exploring further.

Static Pods

This lesson details static pods in Kubernetes, their configuration, and behavior in standalone and clustered environments.

In this lesson, we explore static pods in Kubernetes and how they operate independently of control plane components. Previously, we reviewed the Kubernetes architecture and explained the role of the kubelet as a primary component. Typically, the kubelet receives instructions from the kube-apiserver about which pods to run on its node. These decisions, made by the kube-scheduler and stored in the etcd data store, are standard in a clustered environment. But what happens when the kube-apiserver, kube-scheduler, and other control plane components are absent? How does the node function on its own?

Imagine being isolated at sea with no master or other nodes nearby. Can the kubelet still guide the node? The answer is yes. When the kubelet and a container runtime (such as Docker) are installed directly on the host without a Kubernetes cluster, the kubelet can independently manage the node. In this scenario, because there is no API server to provide pod details, you must supply the pod definition files directly to the kubelet.

Static pods are created directly by the kubelet without the intervention of the API server or other control plane components.

The kubelet is configured to monitor a designated directory on the host where pod definition files are stored. The kubelet periodically scans this directory, reads available files, and creates the corresponding pods. It also monitors these pods and ensures they remain running. For example, if an application crashes, the kubelet restarts it; if an updated file is detected, the kubelet recreates the pod to apply the changes; and if a file is removed, the corresponding pod is also deleted. These pods, created solely by the kubelet, are known as static pods.

It's important to note that only pod-level resources can be created this way. Higher-level abstractions such as ReplicaSets, Deployments, or Services depend on other control plane components (e.g., the replication and deployment controllers) and cannot be managed via the static pod definition files.

Configuring the Static Pods Directory

You can place static pods in any directory on the host. The directory location is provided to the kubelet at startup by using the `--pod-manifest-path` option. Below is an example configuration in the kubelet service file:

```
```bash theme={null} ExecStart=/usr/local/bin/kubelet \ --container-runtime=remote \ --container-runtime-
endpoint=unix:///var/run/containerd/containerd.sock \ --pod-manifest-path=/etc/kubernetes/manifests \ --
kubeconfig=/var/lib/kubelet/kubeconfig \ --network-plugin=cni \ --register-node=true \ --v=2
```

Alternatively, you can specify a configuration file that includes the manifest directory path. For example:

```
```bash theme={null}
# kubelet.service
ExecStart=/usr/local/bin/kubelet \
  --container-runtime=remote \
  --container-runtime-endpoint=unix:///var/run/containerd/containerd.sock \
  --config=kubeconfig.yaml \
  --kubeconfig=/var/lib/kubelet/kubeconfig \
  --network-plugin=cni \
  --register-node=true \
  --v=2
```

```
```yaml theme={null}
```

## kubeconfig.yaml

staticPodPath: /etc/kubernetes/manifests

Many clusters created using kubeadm adopt this approach. When inspecting an existing cluster, first check

You can verify the status of static pods using container runtime commands. For instance, running the Docker

```
```bash theme={null}
docker ps
CONTAINER ID          IMAGE               COMMAND              CREATED              STATUS              PORTS
8e5d4c4db7b6         busybox            "sh -c 'echo Hello K..." 20 seconds ago      Up 20 seconds      0/1
f6737e1149cb         k8s.gcr.io/pause:3.1 "/pause"             24 seconds ago      Up 23 seconds
```

In a standalone scenario, use `docker ps` because the kube-apiserver is not available to process Kubernetes API requests.

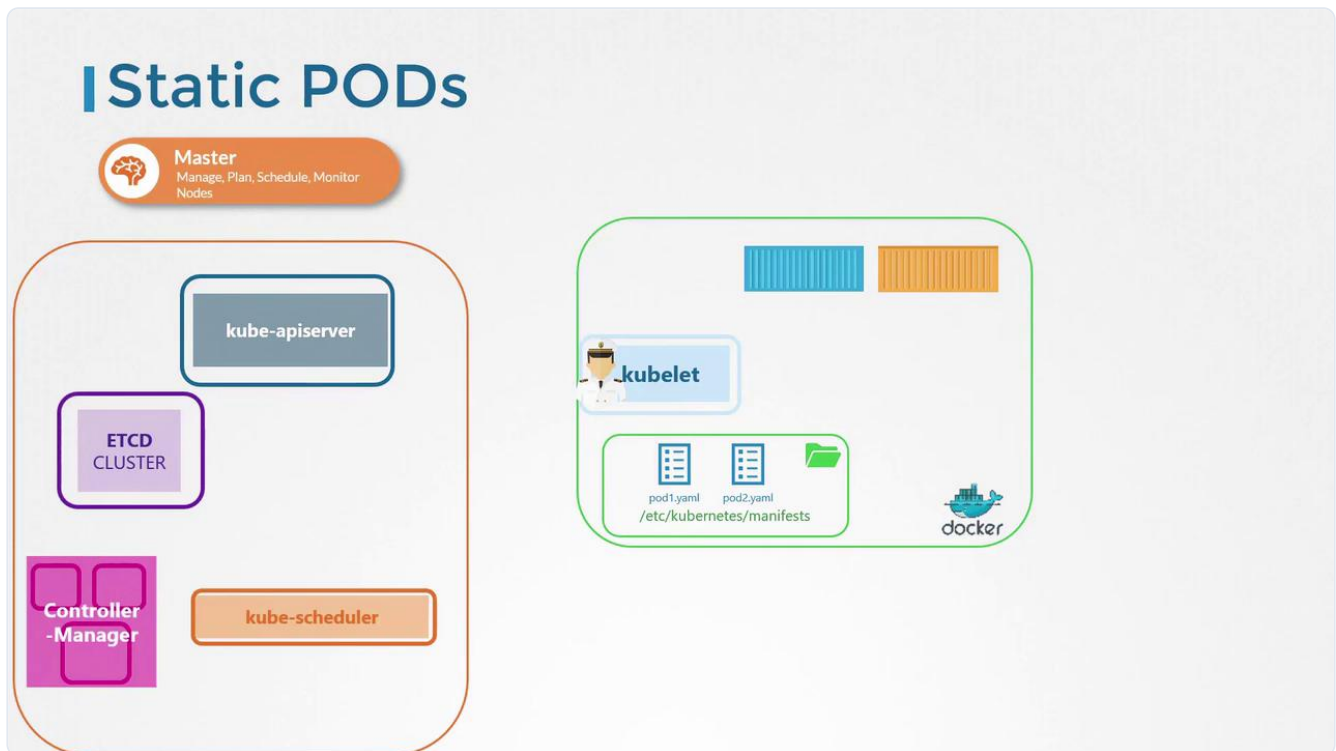
Behavior When Part of a Cluster

When a node is part of a Kubernetes cluster, the kube-apiserver instructs the kubelet to create pods via its HTTP API endpoint. In this mixed mode, the kubelet handles pod definitions provided both from the static pod directory and from the API server. Whenever the kubelet creates a static pod in this configuration, it also creates a mirror object in the kube-apiserver. This mirror object is read-only and can be viewed with `kubectl get pods`, but you cannot modify or delete it through the API. To update a static pod, modify the file in the node's manifest directory.

For example, running the command below on the master node will display the static pod mirror:

```
bash theme={null} kubectl get pods NAME READY STATUS RESTARTS AGE static-web-node01 0/1
ContainerCreating 0 29s
```

Notice that the pod name includes the node name (e.g., "node01") to indicate its origin.



Static Pods vs. DaemonSets

A common question that arises is how static pods differ from DaemonSets. The table below summarizes the key differences between the two:

Feature	Static Pods	DaemonSets
Creation Source	Directly managed by the kubelet	Managed by the DaemonSet controller via the kube-apiserver
Control Plane Involvement	No API server interaction	Requires kube-apiserver communication
Use Case	Typically used for critical control plane components	Ensures a copy of a pod runs on every node
Interaction with Scheduler	Ignored by the kube-scheduler	Ignored by the kube-scheduler

Static pods are especially useful for deploying control plane components themselves. Once the kubelet is installed on all master nodes, you can create pod definition files for essential components like the API server and controller manager. By placing these files in the designated manifest folder, the kubelet ensures they are running as pods and restarts them automatically if they fail. When you check the pods in the kube-system namespace, you'll see these control plane components running as pods—a standard configuration in clusters set up using kubeadm.

Static PODs vs DaemonSets

Static PODs	DaemonSets
Created by the Kubelet	Created by Kube-API server (DaemonSet Controller)
Deploy Control Plane components as Static Pods	Deploy Monitoring Agents, Logging Agents on nodes
Ignored by the Kube-Scheduler	

This lesson has detailed the concept of static pods, their configuration, and their behavior both in standalone and clustered environments. Using static pods is a simplified yet powerful method to manage critical workloads without relying on the full Kubernetes control plane.

That concludes this detailed discussion on static pods. Be sure to review and practice working with static pods in your upcoming tests to reinforce these concepts.

Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

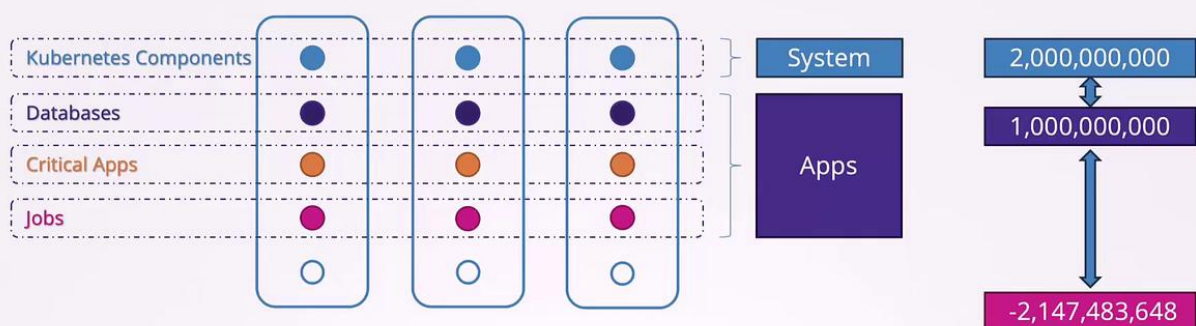
Priority Classes

Understanding priority classes is essential for managing workload scheduling in Kubernetes by assigning numerical values to Pods based on their importance.

Understanding priority classes is essential for managing workload scheduling in Kubernetes. Kubernetes runs various applications as Pods with different levels of importance. For instance, control plane components run within the cluster as Pods and are vital for its operation. Similarly, production databases and critical applications are high-priority while background jobs generally have lower priority. To ensure that more important workloads are scheduled before less critical ones, Kubernetes uses priority classes.

Priority classes allow you to assign a numerical value to Pods, where a higher number indicates higher priority. For user-deployed applications, the value can range from approximately -2 billion to +1 billion. Additionally, there is a reserved range for internal system-critical Pods (like the Kubernetes control plane) which can have values up to 2 billion.

Priorities



note To check the current priority classes in your cluster, run the following command:

```
bash theme={null} kubectl get priorityclass
```

The output may appear as follows:

```
```bash theme={null}
NAME VALUE GLOBAL-DEFAULT AGE PREEMPTIONPOLICY
system-cluster-critical 20000000000 false 7m33s PreemptLowerPriority
system-node-critical 2000010000 false 7m33s PreemptLowerPriority
```

## Creating a New Priority Class

To create a new priority class, define an object with the API version `scheduling.k8s.io/v1`, set the kind to `PriorityClass`, and include metadata with a name, numerical value, and an optional description. For example:

```
```yaml theme={null} apiVersion: scheduling.k8s.io/v1 kind: PriorityClass metadata: name: high-priority value: 1000000000 description: "Priority class for mission critical pods"
```

After creating the priority class, you can assign it to a Pod by specifying the `priorityClassName` field.

Below is an example that demonstrates both the creation of a priority class and how to use it in a Pod definition.

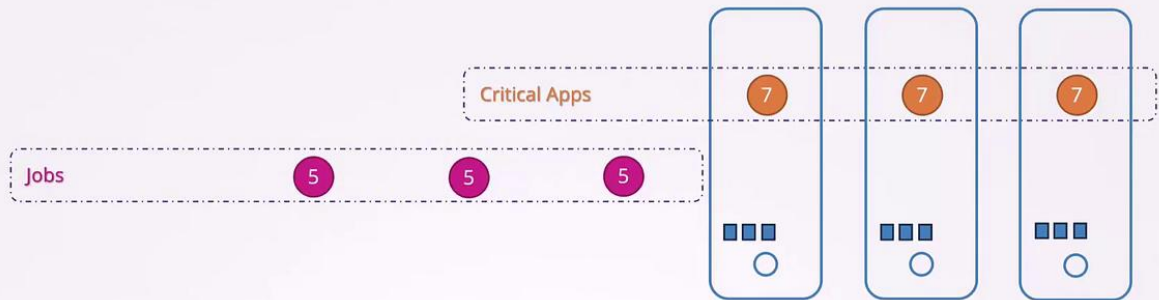
```
```yaml theme={null}
priority-class.yaml
apiVersion: scheduling.k8s.io/v1
kind: PriorityClass
metadata:
 name: high-priority
value: 1000000000
description: "Priority class for mission critical pods"
globalDefault: true

pod-definition.yaml
apiVersion: v1
kind: Pod
metadata:
 name: nginx
 labels:
 app: nginx
spec:
 containers:
 - name: nginx
 image: nginx
 ports:
 - containerPort: 8080
 priorityClassName: high-priority
```

## Pod Priority and Preemption

Consider a scenario where there are two workloads waiting to be scheduled: a critical application with a priority of 7 and a job with a priority of 5. With available resources, the higher priority critical application is scheduled first. If resources remain, the lower priority job is also scheduled.

# Effect of Pod Priority



Now, suppose a new job with a priority of 6 is submitted when no extra resources are available. Whether this new Pod preempts (or evicts) an existing lower priority Pod depends on the preemption policy defined in its priority class. By default, Kubernetes applies the `PreemptLowerPriority` policy, meaning the scheduler will evict lower priority Pods to free up resources for higher priority ones.

The following YAML snippet demonstrates setting the preemption policy to `PreemptLowerPriority`:

```
```yaml theme={null} apiVersion: scheduling.k8s.io/v1 kind: PriorityClass metadata: name: high-priority value: 1000000000 description: "Priority class for mission critical pods" preemptionPolicy: PreemptLowerPriority
```

If you prefer that a higher priority Pod waits for resources rather than preempting lower priority Pods,

```
```yaml theme={null}
apiVersion: scheduling.k8s.io/v1
kind: PriorityClass
metadata:
 name: high-priority
value: 1000000000
description: "Priority class for mission critical pods"
preemptionPolicy: Never
```

Additional Information For more details on Kubernetes scheduling and priority, refer to the [Kubernetes documentation](#).

This concludes the discussion on priority classes. You can now implement these concepts to better manage workload priorities in your Kubernetes cluster.

## Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/lms.txt> Use this file to discover all available pages before exploring further.

# Multiple Schedulers

This article explains how to deploy and configure multiple schedulers in a Kubernetes cluster alongside the default scheduler.

Welcome to this lesson on deploying multiple schedulers in a Kubernetes cluster. In this guide, you will learn how to deploy custom schedulers alongside the default scheduler, configure them correctly, and validate their operation.

Kubernetes' default scheduler distributes pods across nodes evenly while considering factors such as taints, tolerations, and node affinity. However, certain use cases may require a custom scheduling algorithm. For instance, when an application needs to perform extra verification before placing its components on specific nodes, a custom scheduler becomes essential. By writing your own scheduler, packaging it, and deploying it alongside the default scheduler, you can tailor pod placement to your specific needs.

Ensure that every additional scheduler has a unique name. The default scheduler is conventionally named "default-scheduler," and any custom scheduler must be registered with its own distinct name in the configuration files.

## Configuring Schedulers with YAML

Below are examples of configuration files for both the default and a custom scheduler. Each YAML file uses a profiles list to define the scheduler's name.

```
```yaml theme={null}
```

my-scheduler-config.yaml

```
apiVersion: kubescheduler.config.k8s.io/v1 kind: KubeSchedulerConfiguration profiles: - schedulerName: my-scheduler
```

```
```yaml theme={null}
scheduler-config.yaml
apiVersion: kubescheduler.config.k8s.io/v1
kind: KubeSchedulerConfiguration
profiles:
 - schedulerName: default-scheduler
```

---

## Deploying an Additional Scheduler

---

You can deploy an additional scheduler using the existing kube-scheduler binary, tailoring its configuration through specific service files.

### Step 1: Download the kube-scheduler Binary

Begin by downloading the kube-scheduler binary:

```
```bash theme={null} wget https://storage.googleapis.com/kubernetes-release/release/v1.12.0/bin/linux/amd64/kube-scheduler
```

Step 2: Create Service Files

Create separate service files for each scheduler. For example, consider the following definition:

```
```bash theme={null}
kube-scheduler.service
ExecStart=/usr/local/bin/kube-scheduler --config=/etc/kubernetes/config/kube-scheduler.yaml
```

```
```bash theme={null}
```

my-scheduler-2.service

```
ExecStart=/usr/local/bin/kube-scheduler --config=/etc/kubernetes/config/my-scheduler-2-config.yaml
```

Step 3: Define Scheduler Configuration Files

Reference the scheduler names in the associated configuration files:

```
```yaml theme={null}
my-scheduler-2-config.yaml
apiVersion: kubescheduler.config.k8s.io/v1
kind: KubeSchedulerConfiguration
profiles:
 - schedulerName: my-scheduler-2
```

```
```yaml theme={null}
```

my-scheduler-config.yaml

```
apiVersion: kubescheduler.config.k8s.io/v1 kind: KubeSchedulerConfiguration profiles: - schedulerName: my-scheduler
```

<Callout icon="lightbulb" color="#1CB2FE">

Several code blocks might look similar or repeated. The examples above represent a consolidated view for

</Callout>

Deploying the Custom Scheduler as a Pod

In addition to running the scheduler as a service, you can deploy it as a pod inside the Kubernetes cluster.

Example Pod Definition

```
```yaml theme={null}
apiVersion: v1
kind: Pod
metadata:
 name: my-custom-scheduler
 namespace: kube-system
spec:
 containers:
 - name: kube-scheduler
 image: k8s.gcr.io/kube-scheduler-amd64:v1.11.3
 command:
 - kube-scheduler
 - --address=127.0.0.1
 - --kubeconfig=/etc/kubernetes/scheduler.conf
 - --config=/etc/kubernetes/my-scheduler-config.yaml
```

The corresponding custom scheduler configuration file might look like:

```
```yaml theme={null} apiVersion: kubescheduler.config.k8s.io/v1 kind: KubeSchedulerConfiguration profiles: -
schedulerName: my-scheduler
```

<Callout icon="lightbulb" color="#1CB2FE">

Leader election is an important configuration for high-availability environments. It ensures that while

</Callout>

Deploying the Custom Scheduler as a Deployment

In many modern Kubernetes setups—especially those using kubeadm—control plane components run as pods or de

Step 1: Build and Push a Custom Scheduler Image

Create a Dockerfile for your custom scheduler:

```
```dockerfile theme={null}
FROM busybox
ADD ./output/local/bin/linux/amd64/kube-scheduler /usr/local/bin/kube-scheduler
```

Build and push the Docker image:

```
```bash theme={null} docker build -t gcr.io/my-gcp-project/my-kube-scheduler:1.0 . gcloud docker -- push gcr.io/
my-gcp-project/my-kube-scheduler:1.0
```

Step 2: Create ServiceAccount and RBAC Configurations

Prepare the following YAML to create a service account and set appropriate RBAC permissions:

```
```yaml theme={null}
apiVersion: v1
kind: ServiceAccount
metadata:
 name: my-scheduler
 namespace: kube-system

apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
 name: my-scheduler-as-kube-scheduler
subjects:
 - kind: ServiceAccount
 name: my-scheduler
 namespace: kube-system
roleRef:
 kind: ClusterRole
 name: system:kube-scheduler
 apiGroup: rbac.authorization.k8s.io

apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
 name: my-scheduler-as-volume-scheduler
subjects:
 - kind: ServiceAccount
 name: my-scheduler
 namespace: kube-system
roleRef:
 kind: ClusterRole
 name: system:volume-scheduler
 apiGroup: rbac.authorization.k8s.io
```
```

Step 3: Create a ConfigMap for Scheduler Configuration

Define a ConfigMap that includes your custom scheduler configuration:

```
```yaml theme={null} apiVersion: v1 kind: ConfigMap metadata: name: my-scheduler-config namespace: kube-
system data: my-scheduler-config.yaml: | apiVersion: kubescheduler.config.k8s.io/v1beta2 kind:
KubeSchedulerConfiguration profiles: - schedulerName: my-scheduler leaderElection: leaderElect: false
```
```

Step 4: Define the Deployment

Deploy the custom scheduler as a Deployment with the following YAML:

```
```yaml theme={null}
apiVersion: apps/v1
kind: Deployment
metadata:
 name: my-scheduler
 namespace: kube-system
 labels:
 component: scheduler
 tier: control-plane
spec:
 replicas: 1
 selector:
 matchLabels:
 component: scheduler
 tier: control-plane
 template:
 metadata:
 labels:
 component: scheduler
 tier: control-plane
 version: second
 spec:
 serviceAccountName: my-scheduler
 containers:
 - name: kube-second-scheduler
 image: gcr.io/my-gcp-project/my-kube-scheduler:1.0
 command:
 - /usr/local/bin/kube-scheduler
 - --config=/etc/kubernetes/my-scheduler/my-scheduler-config.yaml
 livenessProbe:
 httpGet:
 path: /healthz
 port: 10259
 scheme: HTTPS
 initialDelaySeconds: 15
 readinessProbe:
 httpGet:
 path: /healthz
 port: 10259
 scheme: HTTPS
 volumeMounts:
 - name: config-volume
 mountPath: /etc/kubernetes/my-scheduler
 volumes:
 - name: config-volume
 configMap:
 name: my-scheduler-config
```

Also, ensure a proper ClusterRole exists for the scheduler. For example:

```
```yaml theme={null} apiVersion: rbac.authorization.k8s.io/v1 kind: ClusterRole metadata: name: system:kube-
scheduler annotations: rbac.authorization.kubernetes.io/autoupdate: "true" labels: kubernetes.io/bootstrapping:
```

```
rbac-defaults rules: - apiGroups: - coordination.k8s.io resources: - leases verbs: - create - apiGroups: -
coordination.k8s.io resourceNames: - kube-scheduler - my-scheduler resources: - leases verbs: - get - list -
watch
```

```
***
```

Configuring Workloads to Use the Custom Scheduler

To have specific pods or deployments use your custom scheduler, add the "schedulerName" field in the pod's

```
```yaml theme={null}
apiVersion: v1
kind: Pod
metadata:
 name: nginx
spec:
 containers:
 - name: nginx
 image: nginx
 schedulerName: my-custom-scheduler
```

Deploy this pod with:

```
```bash theme={null} kubectl create -f pod-definition.yaml
```

If the custom scheduler configuration is incorrect, the pod may remain in the Pending state. Conversely,

```
***
```

Verifying Scheduler Operation

To confirm which scheduler assigned a pod, review the events in your namespace:

```
```bash theme={null}
kubectl get events -o wide
```

A sample output might appear as follows:

```
```text theme={null} LAST SEEN COUNT NAME KIND TYPE REASON SOURCE MESSAGE 9s 1 nginx.15 Pod
Normal Scheduled my-custom-scheduler Successfully assigned default/nginx to node01 8s 1 nginx.15 Pod
Normal Pulling kubelet, node01 pulling image "nginx" 2s 1 nginx.15 Pod Normal Pulled kubelet, node01
Successfully pulled image "nginx" 2s 1 nginx.15 Pod Normal Created kubelet, node01 Created container 2s 1
nginx.15 Pod Normal Started kubelet, node01 Started container
```

Notice that the event source is "my-custom-scheduler," confirming that the pod was scheduled by your custom

If you encounter issues, view the scheduler logs with:

```
```bash theme={null}
kubectl logs my-custom-scheduler --namespace=kube-system
```

A sample log output might include messages like:

```
text theme={null} I0204 09:42:25.819338 1 server.go:126] Version: v1.11.3 W0204 09:42:25.822720 1
authorization.go:47] Authorization is disabled W0204 09:42:25.822745 1 authentication.go:55]
Authentication is disabled I0204 09:42:25.822801 1 insecure_serving.go:47] Serving healthz
insecurely on 127.0.0.1:10251 I0204 09:45:14.725407 1 controller_utils.go:1025] Waiting for
caches to sync for scheduler controller I0204 09:45:14.825634 1 controller_utils.go:1032] Caches
are synced for scheduler controller I0204 09:45:14.825814 1 leaderelection.go:185] attempting to
acquire leader lease kube-system/my-custom-scheduler... I0204 09:45:14.834953 1
leaderelection.go:194] successfully acquired lease kube-system/my-custom-scheduler
```

This confirms that the custom scheduler is up and functioning as expected.

---

## Conclusion

---

By following these techniques, you can run both the default Kubernetes scheduler and one or more custom schedulers concurrently. This flexibility allows you to assign specific workloads to the most appropriate scheduler based on your cluster's requirements.

Happy scheduling!

## Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

# Configuring Scheduler Profiles

This article explains configuring scheduler profiles in Kubernetes to customize scheduling behavior and manage multiple profiles within a single scheduler binary.

In this lesson, we dive into the concept of scheduler profiles and their configuration in Kubernetes. We will start with a refresher on how the Kubernetes scheduler functions, illustrated by a simple example where a pod is scheduled to one of several available nodes.

## How Scheduling Works

When a pod is defined, it enters a scheduling queue along with other pending pods. Consider a pod that requires 10 CPU; it will only be scheduled on nodes with at least 10 available CPUs. Additionally, pods with higher priorities are placed at the beginning of the queue. For instance, the following pod definition uses a high-priority class:

```
```yaml theme={null} apiVersion: v1 kind: Pod metadata: name: simple-webapp-color spec: priorityClassName: high-priority containers: - name: simple-webapp-color image: simple-webapp-color resources: requests: memory: "1Gi" cpu: 10
```

Before using this priority, you must create a priority class with a specific name and a priority value. As

Scheduling Phases

After being queued, pods progress through several phases:

1. **Filter Phase:** Nodes that cannot meet the pod's resource requirements (e.g., nodes lacking 10 CPU
2. **Scoring Phase:** Remaining nodes are scored based on resource availability after reserving the req
3. **Binding Phase:** The pod is assigned to the node with the highest score.

Key Scheduler Plugins

Several scheduler plugins play critical roles during these phases:

- * **Priority Sort Plugin:** Sorts pods in the scheduling queue according to their priority.
- * **Node Resources Fit Plugin:** Filters out nodes that do not have the needed resources.
- * **Node Name Plugin:** Checks for a specific node name in the pod specification and filters nodes accord
- * **Node Unschedulable Plugin:** Excludes nodes marked as unschedulable. For instance, running commands is

```
```bash theme={null}
controlplane ~ → kubectl describe node controlplane
Name: controlplane
Roles: control-plane
CreationTimestamp: Thu, 06 Oct 2022 06:19:57 -0400
Taints: node.kubernetes.io/unschedulable:NoSchedule
Unschedulable: true
Lease:
```
```

- * **Scoring Plugins:** During the scoring phase, plugins (such as the Node Resources Fit and Image Localit
- * **Default Binder Plugin:** Finalizes the scheduling process by binding the pod to the selected node.

<Callout icon="lightbulb" color="#1CB2FE">
Kubernetes emphasizes extensibility, allowing you to modify the scheduling process via extension points
</Callout>

The following image outlines the various extension points of the Kubernetes scheduler, including proces

<Frame>
![The image outlines Kubernetes scheduler extension points: Scheduling Queue, Filtering, Scoring, and B
</Frame>

Customizing Scheduler Behavior with Profiles

Rather than running separate scheduler binaries (like default scheduler, MyScheduler, and MyScheduler2) w

Profile Configuration

Each scheduler profile is defined within the scheduler configuration file and behaves like an independent

```
```yaml theme={null}
my-scheduler-2-config.yaml
apiVersion: kubescheduler.config.k8s.io/v1
```

```
kind: KubeSchedulerConfiguration
profiles:
 - schedulerName: my-scheduler-2
 - schedulerName: my-scheduler-3
```

```
```yaml theme={null}
```

my-scheduler-config.yaml

```
apiVersion: kubescheduler.config.k8s.io/v1 kind: KubeSchedulerConfiguration profiles: - schedulerName: my-scheduler
```

```
```yaml theme={null}
scheduler-config.yaml
apiVersion: kubescheduler.config.k8s.io/v1
kind: KubeSchedulerConfiguration
profiles:
 - schedulerName: default-scheduler
```

Each profile allows you to customize plugin configurations. For instance, you could disable specific plugins or enable custom ones. Below is an example where the "my-scheduler-2" profile disables the TaintToleration plugin and enables two custom plugins (MyCustomPluginA and MyCustomPluginB). Additionally, the "my-scheduler-3" profile disables all preScore and score plugins:

```
yaml theme={null} apiVersion: kubescheduler.config.k8s.io/v1 kind: KubeSchedulerConfiguration
profiles: - schedulerName: my-scheduler-2 plugins: score: disabled: - name: TaintToleration
enabled: - name: MyCustomPluginA - name: MyCustomPluginB - schedulerName: my-scheduler-3 plugins:
preScore: disabled: - name: '*' score: disabled: - name: '*' - schedulerName: my-scheduler-4
```

In the plugins section, specify the extension point and then enable or disable plugins by name (or using a wildcard pattern).

This flexible configuration allows you to tailor the scheduling behavior to meet your unique workload requirements by selectively enabling or disabling plugins across different profiles.

## Summary

This lesson provided an overview of Kubernetes scheduling and scheduler profiles. We covered:

- The phases of scheduling: queueing, filtering, scoring, and binding.
- The role of various scheduler plugins and extension points.
- How to configure multiple scheduler profiles within a single scheduler binary to customize scheduling behavior.

For further reading, consider exploring the official documentation on [Kubernetes Scheduling](#) and multi-scheduler profiles.

That's all for this lesson. See you in the next one!

## Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

# 2025 Updates Admission Controllers

This article explores admission controllers in Kubernetes, focusing on their role in enhancing security and enforcing policies before object persistence.

In this lesson, we explore admission controllers in Kubernetes and understand how they enhance security and enforce policies before object persistence in etcd. Every operation performed using the `kubectl` command-line utility—such as creating a pod—is first sent as a request to the API server. The API server then processes the request and stores its information.

When a request reaches the API server, it is first handled by an authentication process. For instance, when using `kubectl`, the required certificates for authentication are provided in the KubeConfig file:

```
```bash theme={null} cat ~/.kube/config apiVersion: v1 clusters: - cluster: certificate-authority-data:
LS0tLS1CRUdJTiBDRVU...
```

After successful authentication, the request undergoes an authorization process using role-based access control (RBAC).

```
```yaml theme={null}
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
 name: developer
rules:
- apiGroups: [""]
 resources: ["pods"]
 verbs: ["list", "get", "create", "update", "delete"]
```

This configuration permits a user assigned to the `developer` role to list, get, create, update, and delete pods. RBAC rules can be further refined to target specific resource names. For instance, to restrict a developer so they can only create pods with designated names:

```
```yaml theme={null} apiVersion: rbac.authorization.k8s.io/v1 kind: Role metadata: name: developer rules: -
apiGroups: [""] resources: ["pods"] verbs: ["create"] resourceNames: ["blue", "orange"]
```

However, object-level permissions may not be sufficient in certain scenarios. When a pod creation request

```
```yaml theme={null}
apiVersion: v1
kind: Pod
metadata:
 name: web-pod
spec:
 containers:
 - name: ubuntu
 image: ubuntu:latest
 command: ["sleep", "3600"]
 securityContext:
 runAsUser: 0
 capabilities:
 add: ["MAC_ADMIN"]
```

Standard RBAC rules operate only at the API level and cannot inspect or modify an object's contents. This limitation is overcome by admission controllers, which validate or even mutate requests prior to persisting objects. Admission controllers can enforce specific policies, such as:

- Changing requests based on internal guidelines.
- Enforcing container image policies.
- Ensuring that certain metadata labels are always applied.

Kubernetes includes a variety of built-in admission controllers. Some common examples include:

- **Always Pull Images:** Forces the pod to pull images from the registry every time.
- **Default Storage Class:** Automatically adds a default storage class to PersistentVolumeClaims (PVCs) when none is provided.
- **Event Rate Limit:** Restricts the API server's request-handling rate.
- **Namespace Exists:** Ensures that requested namespaces exist before proceeding.

## Namespace Admission Controllers

The namespace admission controller ensures that pods are created only in existing namespaces. For example, if you run:

```
```bash theme={null} kubectl run nginx --image nginx --namespace blue
```

and if the "blue" namespace does not exist, you will receive an error like:

```
```bash theme={null}
Error from server (NotFound): namespaces "blue" not found
```

In this situation, after authentication and authorization, the namespace admission controller checks for the existence of the "blue" namespace and rejects the request since it does not exist.

Alternatively, Kubernetes offers the namespace auto-provision admission controller, which automatically creates a namespace if it does not exist (this feature is disabled by default). With the auto-provision controller enabled, executing the same command:

```
```bash theme={null} kubectl run nginx --image nginx --namespace blue
```

results in the automatic creation of the "blue" namespace and a successful pod creation.

To view the admission controllers enabled by default, run:

```
```bash theme={null}
kube-apiserver -h | grep enable-admission-plugins
```

This command will list plugins such as NamespaceLifecycle, LimitRanger, ServiceAccount, TaintNodesByCondition, among others. If you are using a kubeadm-based setup, run the command within the kube-apiserver control plane pod using kubectl exec.

## Enabling Admission Controllers

To add an admission controller, update the `--enable-admission-plugins` flag on the Kube API server. In a kubeadm-based setup, this involves modifying the Kube API server manifest. For example, update the `ExecStart` command in the `systemd` service file as below:

```
```plaintext theme={null} ExecStart=/usr/local/bin/kube-apiserver \ --advertise-address=${INTERNAL_IP} \ --
allow-privileged=true \ --apiserver-count=3 \ --authorization-mode=Node,RBAC \ --bind-address=0.0.0.0 \ --
enable-swagger-ui=true \ --etcd-servers=https://127.0.0.1:2379 \ --event-ttl=1h \ --runtime-config=api/all \ --
service-cluster-ip-range=10.32.0.0/24 \ --service-node-port-range=30000-32767 \ --v=2 \ --enable-admission-
plugins=NodeRestriction,NamespaceAutoProvision
```

For kubeadm-based setups where the API server runs as a pod, the manifest might look like this:

```
```yaml theme={null}
apiVersion: v1
kind: Pod
metadata:
 creationTimestamp: null
 name: kube-apiserver
 namespace: kube-system
spec:
 containers:
 - command:
 - kube-apiserver
 - --authorization-mode=Node,RBAC
 - --advertise-address=172.17.0.107
 - --allow-privileged=true
 - --enable-bootstrap-token-auth=true
 - --enable-admission-plugins=NodeRestriction,NamespaceAutoProvision
 image: k8s.gcr.io/kube-apiserver-amd64:v1.11.3
 name: kube-apiserver
```

To disable specific admission controller plugins, use the `--disable-admission-plugins` flag similarly.

After updating your configuration, running the following command in a non-existent namespace:

```
```bash theme={null} kubectl run nginx --image nginx --namespace blue
```

should output:

```
```bash theme={null}  
Pod/nginx created!
```

Verifying the available namespaces:

```
```bash theme={null} kubectl get namespaces
```

will display:

```
```bash theme={null}  
NAME STATUS AGE
blue Active 3m
default Active 23m
kube-public Active 24m
kube-system Active 24m
```

This demonstrates how admission controllers not only reject invalid requests but can also perform backend operations like automatically creating a namespace.

Both the namespace auto-provision and namespace existence admission controllers are deprecated. They have been replaced by the namespace lifecycle admission controller, which enforces that requests to non-existent namespaces are rejected and protects default namespaces (default, kube-system, and kube-public) from deletion.

That's it for now. Practice using admission controllers in your exercises to reinforce your learnings. See you in the next lesson!

## Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/lms.txt> Use this file to discover all available pages before exploring further.

# 2025 Updates Validating and Mutating Admission Controllers

This article explores Kubernetes Admission Controllers, detailing validating and mutating types, their configurations, and how to implement custom external admission webhooks.

This article provides an in-depth look at Kubernetes Admission Controllers, focusing on both validating and mutating types. Learn how these controllers work, inspect typical API requests, and explore how to configure custom external admission webhooks for advanced validations and mutations.

## Validating Admission Controllers

Validating admission controllers verify that an object meets specific criteria before it is persisted in the cluster. For example, the namespace existence or namespace lifecycle admission controller checks if a namespace exists and, if not, rejects the incoming request. Another example is the default storage class admission controller. When you create a PersistentVolumeClaim (PVC) without specifying a storage class, this validating controller intercepts the request and modifies it by adding the default storage class.

Consider the initial PVC creation request:

```
```yaml theme={null} apiVersion: v1 kind: PersistentVolumeClaim metadata: name: myclaim spec: accessModes:
- ReadWriteOnce resources: requests: storage: 500Mi
```

After the request passes through authentication, authorization, and the admission controller, the modified request is as follows:

```
```yaml theme={null}
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
 name: myclaim
spec:
 accessModes:
 - ReadWriteOnce
 resources:
 requests:
 storage: 500Mi
 storageClassName: default
```

Since the controller modifies the request, such controllers are also considered mutating admission controllers.

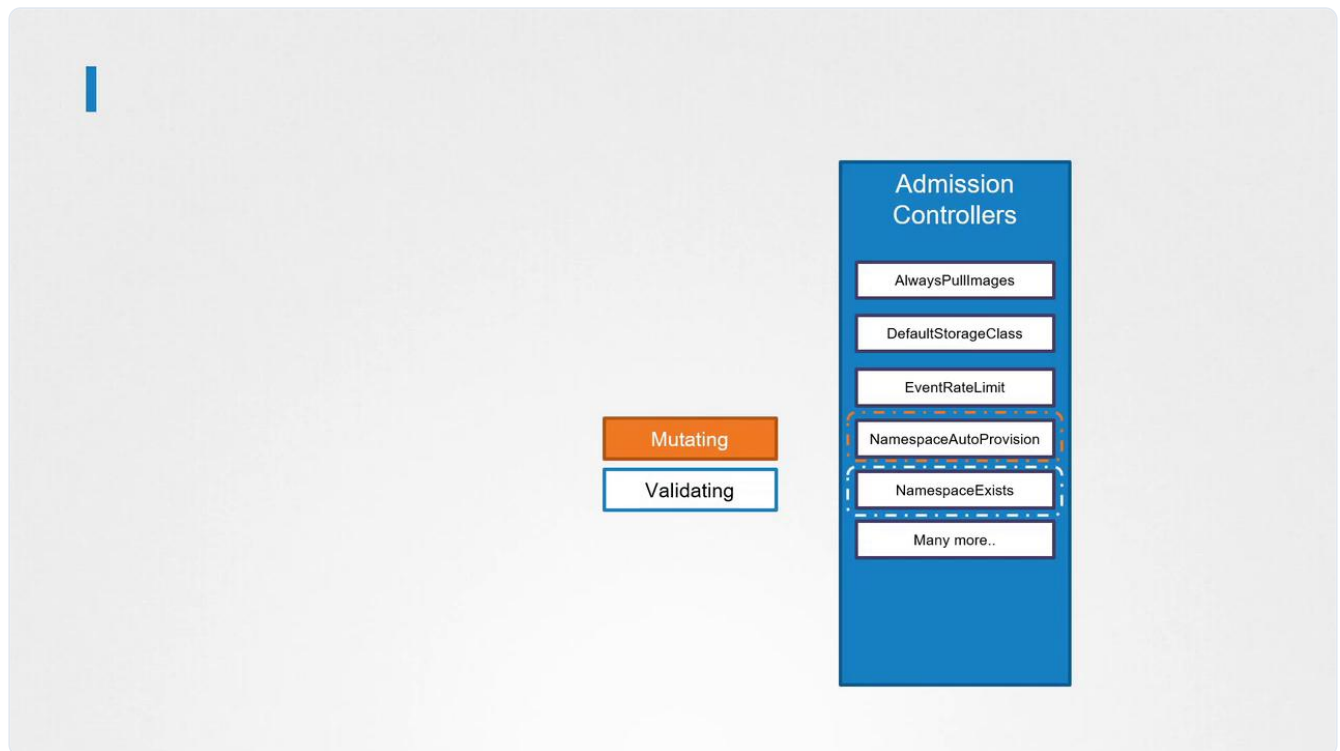
## Mutating vs. Validating Admission Controllers

Admission controllers in Kubernetes can be classified into two main types:

- **Mutating Admission Controllers:** Modify (mutate) objects before they are persisted.
- **Validating Admission Controllers:** Validate objects to ensure they meet specific criteria, allowing or denying the request accordingly.

Some controllers perform both mutation and validation. Typically, mutating controllers run first so that subsequent validating controllers can work with the modified object. For instance, if a namespace auto-provisioning mutating admission controller creates a missing namespace before the namespace existence validating controller runs, the request proceeds smoothly. However, if the validating controller executes first, it would reject the request due to the missing namespace.

If any admission controller (mutating or validating) rejects a request, the entire request is denied and an error is returned to the user.



All built-in admission controllers are part of the Kubernetes source code. But for custom validations and mutations, Kubernetes supports external admission controllers using webhook mechanisms:

- Mutating Admission Webhook
- Validating Admission Webhook

## Configuring External Admission Webhooks

External admission webhooks can point to servers either inside or outside the Kubernetes cluster. Once the built-in admission controllers finish processing, the API server sends an AdmissionReview object (in JSON format) containing request details such as user information, operation type, and object metadata to the external webhook.

Below is an example AdmissionReview request sent to a webhook server:

```
```json theme={null} { "apiVersion": "admission.k8s.io/v1", "kind": "AdmissionReview", "request": { "uid": "705ab415-6393-11e7-b7cc-4201a8000002", "kind": {"group": "autoscaling", "version": "v1", "kind": "Scale"}, "resource": {"group": "apps", "version": "v1", "resource": "deployments"}, "subresource": "scale", "requestKind": {"group": "autoscaling", "version": "v1", "kind": "Scale"}, "requestResource": {"group": "apps", "version": "v1", "resource": "deployments"} } }
```

The webhook server processes the AdmissionReview request and responds with its own AdmissionReview JSON object.

```
```json theme={null}
{
 "apiVersion": "admission.k8s.io/v1",
 "kind": "AdmissionReview",
 "request": {
 "uid": "705ab41f-6393-11e8-b7cc-4201a8000002",
 "kind": {
 "group": "autoscaling",
 "version": "v1",
 "kind": "Scale"
 },
 },
 "resource": {
 "group": "apps",
 "version": "v1",
 "resource": "deployments"
 },
 "subResource": "scale",
 "requestKind": {
 "group": "autoscaling",
 "version": "v1",
 "kind": "Scale"
 },
 "requestResource": {
 "group": "apps",
 "version": "v1",
 "resource": "deployments"
 }
},
"response": {
 "uid": "<value_from request.uid>",
 "allowed": true
}
}
```

If the "allowed" field is set to false, the API server rejects the request.

## Deploying Your Admission Webhook Server

---

To implement your custom webhook server, you must deploy it and ensure it can support the required mutation and validation APIs by returning correct JSON responses. The following Go code snippet illustrates a basic webhook server setup:

```
```go theme={null} package main
```

```
import ( "encoding/json" "flag" "fmt" "io/ioutil" "net/http" "k8s.io/api/admission/v1beta1" metav1 "k8s.io/apimachinery/pkg/apis/meta/v1" "k8s.io/klog" )
```

```
// toAdmissionResponse creates an AdmissionResponse with an error message. func toAdmissionResponse(err error) v1beta1.AdmissionResponse { return v1beta1.AdmissionResponse{ Result: &metav1.Status{ Message: err.Error(), }, } }
```

```
// admitFunc defines the signature for validators and mutators. type admitFunc func(v1beta1.AdmissionReview) v1beta1.AdmissionResponse
```

```
// serve processes the HTTP request before calling the admitFunc. func serve(w http.ResponseWriter, r *http.Request, admit admitFunc) { var data []byte if r.Body == nil { return } data, err := ioutil.ReadAll(r.Body) if err != nil { return } // Additional processing logic goes here... }
```

Below is a pseudocode example of a webhook server implemented in Python. This example demonstrates both validating and mutating admission controllers.

Python Webhook Server Example

In this Python example, the `/validate` endpoint rejects requests where the object's name matches the user's name.

```
```python
theme={null}
from flask import Flask, request, jsonify
import base64

app = Flask(__name__)

@app.route("/validate", methods=["POST"])
def validate():
 object_name = request.json["request"]["object"]["metadata"]["name"]
 user_name = request.json["request"]["userInfo"]["name"]
 status = True
 message = ""
 if object_name == user_name:
 message = "You can't create objects with your own name"
 status = False
 return jsonify(
 {
 "response": {
 "allowed": status,
 "uid": request.json["request"]["uid"],
 "status": {"message": message},
 }
 }
)

@app.route("/mutate", methods=["POST"])
def mutate():
 user_name = request.json["request"]["userInfo"]["name"]
 patch = [{"op": "add", "path": "/metadata/labels/users", "value": user_name}]
 encoded_patch = base64.b64encode(str(patch).encode()).decode()
 return jsonify(
 {
 "response": {
 "allowed": True,
 "uid": request.json["request"]["uid"],
 "patch": encoded_patch,
 "patchType": "JSONPatch",
 }
 }
)

if __name__ == "__main__":
 app.run(port=443, debug=True)
```
```

Even if you may not need to write such code for an exam, understanding the flow and structure of AdmissionReview objects is essential for managing custom admission controllers in Kubernetes.

Hosting the Webhook Server and Configuring Webhooks

After developing your webhook server, deploy it either as a standalone service or containerize it and run it within your Kubernetes cluster. If running inside the cluster, ensure that the server is accessible via a Kubernetes Service.

To instruct the API server to use your webhook for validations or mutations, create a `ValidatingWebhookConfiguration` or a `MutatingWebhookConfiguration` object. Below is an example configuration for a validating webhook that triggers on pod creation:

```
yaml      theme={null}      apiVersion:      admissionregistration.k8s.io/v1      kind:
ValidatingWebhookConfiguration metadata: name: "pod-policy.example.com" webhooks: - name: "pod-
policy.example.com" clientConfig: service: namespace: "webhook-namespace" name: "webhook-service"
caBundle: "Ci0tLS0tQk.....tLS0K" rules: - apiGroups: [""] apiVersions: ["v1"] operations:
["CREATE"] resources: ["pods"] scope: "Namespaced"
```

In the configuration above:

- The **clientConfig** block specifies how the API server connects to your webhook service, including the TLS certificate bundle (`caBundle`).
- The **rules** section defines the operations that trigger the webhook—in this case, whenever a pod is created.

Once this configuration is applied, the Kubernetes API server will call your webhook service for each relevant pod creation event, and the request will be allowed or rejected according to the logic implemented in your webhook.

Conclusion

This guide has provided an overview of Kubernetes admission controllers—both validating and mutating—and demonstrated how to configure external admission webhooks. By applying these concepts, you can enhance the security, flexibility, and management of your Kubernetes clusters.

For further reading, check out these resources:

- [Kubernetes Documentation](#)
- [Kubernetes Basics](#)
- [Terraform Registry](#)

Experiment with these configurations and webhook implementations in a test environment to gain hands-on experience and further solidify your understanding of Kubernetes Admission Controllers.